Laboratorio de Programación y Estructuras de Datos (PRD) – Sesión 3: Estructuras de datos: structs, vectores y matrices

En esta sesión de laboratorio trabajaremos distintas estructuras de datos en lenguaje C: *structs*, vectores y matrices. Como en las sesiones anteriores, se proporciona junto a este enunciado un proyecto Netbeans parcialmente implementado, que deberéis completar con vuestras soluciones a los distintos ejercicios propuestos.

1. Desarrollo de la práctica

Ejercicio 1:

Dado el siguiente tipo de dato `t_fraccion` que almacena la información de una fracción:

```
typedef struct
{
  int num;            /* Numerador   */
  int den;            /* Denominador */
}t_fraccion;
```

Escriba un programa en C (función `sesion3_ej1()` en *ejercicios.c)*, que sea capaz de leer desde teclado dos fracciones y muestre por pantalla el resultado de sumarlas, restarlas, multiplicarlas y dividirlas.

Nota: No es necesario simplificar las fracciones.

Ejemplos de ejecución (en negrita los datos introducidos por el usuario):

```
Fraccion 1 (formato x/y): 1/2
Fraccion 2 (formato x/y): 7/-2

RESULTADO DE LAS OPERACIONES:

Suma: 1/2 + 7/-2 = 12/-4

Resta: 1/2 - 7/-2 = -16/-4

Multiplicacion: 1/2 * 7/-2 = 7/-4

Division: 1/2 / 7/-2 = -2/14
```

```
Fraccion 1 (formato x/y): 3/4
Fraccion 2 (formato x/y): -7/2

RESULTADO DE LAS OPERACIONES:

Suma: 3/4 + -7/2 = -22/8

Resta: 3/4 - -7/2 = 34/8

Multiplicacion: 3/4 * -7/2 = -21/8

Division: 3/4 / -7/2 = 6/-28
```

Ejercicio 2:

Dado el siguiente tipo de dato `t_carta` que almacena la información de una carta de la baraja española:

```
typedef struct
{
    char fig;    /* Figura: 'R' (Rey), 'C' (Caballo), 'S' (Sota),
                 *         '9','8','7','6','5','4','3','2','1' */

    char pal;    /* Palo: 'O' (Oros), 'C' (Copas), 'E' (Espadas),
                 *       'B' (Bastos) */
}t_carta;
```

a) Escriba un programa en C (función `sesion3_ej2a()` en *ejercicios.c)* que lea desde teclado (como caracteres) la figura y el palo de una carta, almacene la información en una variable de tipo `t_carta` y finalmente, la muestre por pantalla. El formato de entrada y salida del programa se detalla en los ejemplos de ejecución que se muestran a continuación (en negrita se muestran los datos introducidos por el usuario):

```
Figura carta: r o R, c o C, s o S, 9, ... , 1
Palo carta: o o O(OROS), c o C(COPAS), e o E(ESPADAS) o b o B(BASTOS)
Carta (figura,palo): (r,c)

(R,C)
```

```
Figura carta: r o R, c o C, s o S, 9, ... , 1
Palo carta: o o O(OROS), c o C(COPAS), e o E(ESPADAS) o b o B(BASTOS)
Carta (figura,palo): (6,O)

(6,O)
```

```
Figura carta: r o R, c o C, s o S, 9, ... , 1
Palo carta: o o O(OROS), c o C(COPAS), e o E(ESPADAS) o b o B(BASTOS)
Carta (figura,palo): (C,B)

(C,B)
```

```
Figura carta: r o R, c o C, s o S, 9, ... , 1
Palo carta: o o O(OROS), c o C(COPAS), e o E(ESPADAS) o b o B(BASTOS)
Carta (figura,palo): (1,e)

(1,E)
```

Nota: Observe que al mostrar la carta por pantalla el palo se imprime en colores, mostrando diferente color según el palo (color rojo para copas, amarillo para oros, verde para bastos y azul para espadas). Utilice la librería `colours.h` para mostrar el palo de la carta en el color que corresponde.

b) Extienda el código anterior (función `sesion3_ej2b()` en *ejercicios.c*) e incluya todas las sentencias que considere necesarias para que el programa lea del teclado una secuencia de cartas que finalizan con el carácter punto y determine cuál de las cartas introducidas es la mayor, suponiendo que: OROS > COPAS > ESPADAS > BASTOS

y, en caso de tener el mismo palo, 12 (Rey) > 11 (Caballo) > 10 (Sota) > 9 > ... > 1. Finalmente, el programa debe mostrar por pantalla la carta.

Nota: Suponga que la secuencia de cartas no está vacía y que como mínimo tiene 1 carta.

Ejemplos de ejecución (en negrita los datos introducidos por el usuario):

```
Figura carta: r o R, c o C, s o S, 9, ... , 1

Palo carta: o o O(OROS), c o C(COPAS), e o E(ESPADAS) o b o B(BASTOS)

Introduzca cartas (fig,palo),..,(fig,palo). : (3,o),(8,o),(8,B),(R,o).

La carta mayor es: (R,O)
```

```
Figura carta: r o R, c o C, s o S, 9, ... , 1

Palo carta: o o O(OROS), c o C(COPAS), e o E(ESPADAS) o b o B(BASTOS)

Introduzca cartas (fig,palo),..,(fig,palo). : (5,b),(c,e),(1,c),(c,b).

La carta mayor es: (1,C)
```

Ejercicio 3:

Dado el siguiente tipo de dato `t_estudiante`:

```
typedef struct
{
    int id;
    float nota_promedio;
    int fecha_nacimiento;          /* Formato: aaaammdd */
    int num_asignaturas_aprobadas;
}t_estudiante;
```

Escriba un programa en C (función `sesion3_ej3()` en *ejercicios.c)* que declare un vector de datos de tipo `t_estudiante`, de `NUM_ESTUDIANTES` (constante a definir, con valor igual a 3) posiciones. El programa debe permitir rellenar dicho vector proporcionando los datos de cada estudiante (id, nota promedio, fecha de nacimiento y número de asignaturas aprobadas) a través del teclado. Además, el programa debe mostrar por pantalla:

1. Los datos del estudiante con mayor número de asignaturas aprobadas. Considere que puede haber en la secuencia varios estudiantes que tengan el máximo de asignaturas aprobadas. En este caso, el programa debe mostrar los datos del primer estudiante en la secuencia que tiene el mayor número de asignaturas aprobadas.
2. El número total de estudiantes que tienen nota promedio excelente (mayor o igual a 9.0) y que tienen más de 5 asignaturas aprobadas.

Ejemplo de ejecución (en negrita los datos introducidos por el usuario):

```
Id del estudiante: 3
Nota promedio: 8.76
Fecha nacimiento (aaaammdd): 19850312
Total de asignaturas aprobadas: 6

Id del estudiante: 5
Nota promedio: 6.12
Fecha nacimiento (aaaammdd): 19841111
Total de asignaturas aprobadas: 4

Id del estudiante: 6
Nota promedio: 9.12
Fecha nacimiento (aaaammdd): 19841010
Total de asignaturas aprobadas: 6


Estudiante con mas asignaturas aprobadas:
Id: 3
Nota promedio: 8.76
Fecha nacimiento: 19850312
Total de asignaturas aprobadas: 6

Total de estudiantes con nota >= 9.0 y mas de 5 asignaturas
aprobadas: 1
```

Ejercicio 4:

Se desea mantener la información de distancias en kilómetros entre 5 capitales según la tabla siguiente:

	Londres	Madrid	París	Roma	Viena
Londres	0	1720	456	1845	1473
Madrid	1720	0	1272	1965	2399
París	456	1272	0	1468	1280
Roma	1845	1965	1468	0	1130
Viena	1473	2399	1280	1130	0

Para representar dicha tabla en C, se utilizará una matriz cuadrada de 5 filas x 5 columnas. Según la tabla anterior cada capital tiene asociado un identificador entre 0 y 4 (Londres el 0, Madrid el 1, Paris el 2, Roma el 3 y Viena el 4).
En la siguiente declaración, la variable `distancias` representará dicha matriz:

```
#define MAX_CAPITALES 5
int distancias[MAX_CAPITALES][MAX_CAPITALES];
```

a) Implementar un código (función `sesion3_ej4a()` en *ejercicios.c*) en donde se rellene las diferentes posiciones de la matriz distancias con la información de distancias en km proporcionada en la tabla anterior. Además, se debe añadir al fichero el código necesario para que lea del teclado dos identificadores de capitales (valor entre 0 y `MAX_CAPITALES-1`) y muestre por pantalla la distancia entre ambas capitales. El programa debe controlar que los identificadores de capitales están dentro del rango permitido.

Ejemplos de ejecución (en negrita los datos introducidos por el usuario):

```
Introduce el identificador de las dos capitales (id1, id2): 5,6

Alguno de los identificadores es invalido.

Introduce el identificador de las dos capitales (id1, id2): 3,4

La distancia entre las capitales 3 y 4 es 1130
```

```
Introduce el identificador de las dos capitales (id1, id2): 1,3

La distancia entre las capitales 1 y 3 es 1965
```

```
Introduce el identificador de las dos capitales (id1, id2): 4,1

La distancia entre las capitales 4 y 1 es 2399
```

b) Se define un camino como una secuencia de identificadores de capitales, donde el primer elemento de la secuencia indica el origen, los siguientes las capitales por las que se pasa y el último el destino. Por ejemplo, un camino con origen en París que pasa por Viena y finaliza en Londres sería la secuencia {2, 4, 0}.

Implementar un código (función `sesion3_ej4b()` en *ejercicios.c*) que sea capaz de leer un camino del teclado y mostrar por pantalla el total de kilómetros que tiene. El camino introducido acabará con el entero -1. El programa debe controlar que los identificadores de todas las ciudades del camino sean válidos.

Ejemplos de ejecución (en negrita los datos introducidos por el usuario):

```
Introduce el id de las ciudades a visitar (id1, id2, ...): 2,3,-1

La distancia del camino es: 1468
```

```
Introduce el id de las ciudades a visitar (id1, id2, ...): 5,6,-1

Alguno de los identificadores es invalido.
La distancia del camino es: 0
```

```
Introduce el id de las ciudades a visitar (id1, id2, ...): 2,-1

La distancia del camino es: 0
```

```
Introduce el id de las ciudades a visitar (id1, id2, ...):
2,3,4,0,-1

La distancia del camino es: 4071
```